Événements liés au clavier

Pour gérer les événements liés au clavier en JavaScript, on utilise principalement les événements `keydown` et `keyup` . Voici une brève description de chacun :

- `keydown` : Cet événement est déclenché lorsque la touche est enfoncée.
- `keyup` : Cet événement est déclenché lorsque la touche est relâchée.

De la même manière que pour les événements de la souris, on peut attacher des gestionnaires d'événements aux éléments HTML pour réagir aux actions de l'utilisateur.Généralement, on attache ces événements à l'élément `document`

```
document.addEventListener("keydown", enfoncerTouche);

function enfoncerTouche(evenement) {
    console.log("Touche enfoncée : " + evenement.key);
}
```

Récupérer la touche enfoncée

Le paramètre `evenement` passé à la fonction de gestion d'événement contient des informations sur l'événement, y compris la touche enfoncée. On peut accéder à cette information via la propriété `key` de l'objet événement. La valeur retournée est une chaîne de caractères représentant la touche enfoncée, par exemple "a", "Enter", "ArrowUp", etc.

```
document.addEventListener("keydown", enfoncerTouche);

function enfoncerTouche(evenement) {
    const touche = evenement.key;
    console.log("Touche enfoncée : " + touche);
}
```

Les touches spéciales sont également reconnues et peuvent être utilisées pour des interactions plus complexes.

- `Enter` : Touche Entrée
- `Escape` : Touche Échap
- `ArrowUp` : Flèche Haut
- `ArrowDown` : Flèche Bas
- `ArrowLeft` : Flèche Gauche
- `ArrowRight` : Flèche Droite
- `Shift` : Touche Maj
- `Control` : Touche Ctrl
- `Alt` : Touche Alt
- `Tab` : Touche Tabulation

- `Backspace` : Touche Retour arrière

Empêcher le comportement par défaut du navigateur

Dans certains cas, certaines touches ont des comportements par défaut dans le navigateur (ex: espace pour faire défiler la page, tab pour changer de champ de formulaire, etc.). Pour éviter ces comportements indésirables dans un jeu, on utilise la méthode `preventDefault()` sur l'objet événement.

```
document.addEventListener("keydown", function (evenement) {
    // Dire au navigateur : "N'exécute pas ton comportement normal"
    evenement.preventDefault();

    if (evenement.key === " ") {
        // Maintenant, seul notre code de jeu s'exécute
        joueur.sauter();
    }
});
```

Exemple : Déplacer un élément avec les flèches du clavier

Voici un exemple simple où l'on déplace une image sur la page en utilisant les touches fléchées du clavier avec la balise `<canvas>`.

```
let oCanvasHTML = document.querySelector("canvas");
let oContexte = oCanvasHTML.getContext("2d");
let nLargeur = oCanvasHTML.width;
let nHauteur = oCanvasHTML.height;

//Dans cet exemple, on utilise un objet pour représenter la voiture (Voir les
notes sur les objets)
let voiture = {
    positionX: 100,
    positionY: 100,
    largeur: 50,
    hauteur: 30,
    vitesse: 5,
    image: new Image(),
    imageSrc: "images/voiture.png",
};
voiture.image.src = voiture.imageSrc;

function initialiser() {
    setInterval(boucleJeu, 1000 / 60);
    document.addEventListener("keydown", onClicCanvas);
}

function boucleJeu() {
    oContexte.clearRect(0, 0, nLargeur, nHauteur);
    oContexte.drawImage(voiture.image, voiture.positionX, voiture.positionY);
}
```

```
function onClicCanvas(evenement) {
    let touche = evenement.key;

    if (touche == "ArrowUp") {
        voiture.positionY -= voiture.vitesse;
    } else if (touche == "ArrowDown") {
        voiture.positionY += voiture.vitesse;
    } else if (touche == "ArrowLeft") {
        voiture.positionX -= voiture.vitesse;
    } else if (touche == "ArrowRight") {
        voiture.positionX += voiture.vitesse;
    }
}

window.addEventListener("load", initialiser);
```

Exemple : Attendre la réponse de l'utilisateur avec les états de jeu

Dans un jeu où l'on doit attendre une réponse écrite par l'utilisateur, on peut utiliser les événements liés au clavier pour capturer la saisie. Voici un exemple simple où l'on demande à l'utilisateur d'appuyer sur une touche spécifique pour continuer.

```
let oCanvasHTML = document.querySelector("canvas");
let oContexte = oCanvasHTML.getContext("2d");

let nLargeur = oCanvasHTML.width;
let nHauteur = oCanvasHTML.height;

let sEtatJeu = "attenteReponse"; // états possibles : "intro", "attenteReponse",
"validation", "fin"

let reponseUtilisateur = "";
let motATrouver = "javascript";

function initialiser() {
    setInterval(boucleJeu, 1000 / 60);
    document.addEventListener("keydown", enfoncerTouche);
}

function boucleJeu() {
    oContexte.clearRect(0, 0, nLargeur, nHauteur);

    if (sEtatJeu == "attenteReponse") {
        afficherJeu();
    } else if (sEtatJeu == "fin") {
        oContexte.font = "30px Arial";
        oContexte.fillStyle = "green";
        oContexte.fillText("Bravo ! Vous avez trouvé le mot : " + motATrouver, 50,
nHauteur / 2);
    }
}
```

```
function afficherJeu() {
    oContexte.font = "20px Arial";
    oContexte.fillStyle = "black";
    oContexte.fillText("Tapez le mot : " + motATrouver, 50, nHauteur / 2 - 30);
    oContexte.fillText("Votre réponse : " + reponseUtilisateur, 50, nHauteur / 2);
    oContexte.fillText("Appuyez sur 'Enter' pour valider", 50, nHauteur / 2 + 30);
}

function enfoncerTouche(evenement) {
    let touche = evenement.key;

    if (sEtatJeu == "attenteReponse") {
        if (touche == "Enter") {
            validerReponse();
            sEtatJeu = "validation";
        } else if (touche == "Backspace") {
            // On retire le dernier caractère
            reponseUtilisateur = reponseUtilisateur.slice(0, -1);
            console.log("Réponse actuelle : " + reponseUtilisateur);
        } else if (touche.length == 1) {
            // Vérifie si c'est une touche de caractère
            reponseUtilisateur += touche;
            console.log("Réponse actuelle : " + reponseUtilisateur);
        }
    }
}

function validerReponse() {
    if (reponseUtilisateur.toLowerCase() == motATrouver) {
        console.log("Bonne réponse !");
        sEtatJeu = "termine";
    } else {
        console.log("Mauvaise réponse, essayez encore.");
        reponseUtilisateur = "";
    }
}
```

Contrôles avancés : Différence keydown vs keyup

Comprendre quand utiliser chaque événement est crucial :

```
let joueur = { x: 100, y: 100, enMarche: false, peutTirer: true };

// KEYDOWN - Pour les actions continues (mouvement)
document.addEventListener("keydown", function (evenement) {
    if (evenement.key === "ArrowLeft" && !joueur.enMarche) {
        joueur.enMarche = true;
        console.log("Commence à marcher vers la gauche");
    }

    // Empêcher la répétition automatique du navigateur
    if (evenement.key === " ") {
```

```
        evenement.preventDefault(); // Empêche le scroll de la page
    }
});

// KEYUP - Pour les actions ponctuelles (tir, saut)
document.addEventListener("keyup", function (evenement) {
    if (evenement.key === "ArrowLeft") {
        joueur.enMarche = false;
        console.log("Arrête de marcher");
    }

    // Tir au relâchement (pour éviter la répétition)
    if (evenement.key === " " && joueur.peutTirer) {
        tirerProjectile();
        joueur.peutTirer = false;

        // Cooldown de tir
        setTimeout(function () {
            joueur.peutTirer = true;
        }, 500); // 0.5 seconde entre chaque tir
    }
});
```

Gestion de plusieurs touches simultanées

Dans les jeux, il est souvent nécessaire de détecter plusieurs touches enfoncées en même temps (ex: flèche + espace pour sauter en courant). On utilisera alors un objet pour suivre l'état de chaque touche (enfoncée ou relâchée) au lieu d'avoir une seule variable. En utilisant la notation par crochets, on ajoute une propriété en utilisant une variable. On ne sait pas à l'avance quelles touches seront enfoncées, donc on utilise cette méthode dynamique.

```
// Objet pour suivre l'état de chaque touche
let touchesEnfoncees = {};

function initialiser() {
    setInterval(boucleJeu, 1000 / 60);

    // Détecter quand une touche est enfoncée
    document.addEventListener("keydown", function (evenement) {
        //Ex: si la touche "ArrowLeft" est enfoncée, on crée une propriété
"ArrowLeft" dans l'objet touchesEnfoncees et on lui assigne la valeur true
        touchesEnfoncees[evenement.key] = true;
    });

    // Détecter quand une touche est relâchée
    document.addEventListener("keyup", function (evenement) {
        //Ex: si la touche "ArrowLeft" est relâchée, on crée une propriété
"ArrowLeft" dans l'objet touchesEnfoncees et on lui assigne la valeur false
        touchesEnfoncees[evenement.key] = false;
    });
}
```

```
let joueur = {
    x: 100,
    y: 100,
    vitesse: 3,
    vitesseSaut: 0,
    largeur: 30,
    hauteur: 30,
    surSol: true,
};

function boucleJeu() {
    oContexte.clearRect(0, 0, nLargeur, nHauteur);

    // Mouvement horizontal continu
    if (touchesEnfoncees["ArrowLeft"] == true) {
        joueur.x -= joueur.vitesse;
    }
    if (touchesEnfoncees["ArrowRight"] == true) {
        joueur.x += joueur.vitesse;
    }

    // Saut (seulement si sur le sol)
    if (touchesEnfoncees[" "] == true && joueur.surSol == true) {
        // Espace = saut
        joueur.vitesseSaut = -8; // Vitesse négative = vers le haut
        joueur.surSol = false;
    }

    // Action spéciale (tir) - combinaison
    if (touchesEnfoncees["ArrowUp"] == true && touchesEnfoncees[" "] == true) {
        tirerProjectile(joueur.x, joueur.y);
    }

    // Appliquer la gravité pour le saut
    if (!joueur.surSol == true) {
        joueur.y += joueur.vitesseSaut;
        joueur.vitesseSaut += 0.5; // Gravité

        // Retomber au sol
        if (joueur.y >= 100) {
            joueur.y = 100;
            joueur.surSol = true;
            joueur.vitesseSaut = 0;
        }
    }

    // Contraintes d'écran, si le joueur sort de l'écran, le remettre à
l'intérieur
    if (joueur.x < 0) {
        joueur.x = 0;
    }

    if (joueur.x > nLargeur - joueur.largeur) {
        joueur.x = nLargeur - joueur.largeur;
    }
```

```
    // Dessiner le joueur
    oContexte.fillStyle = "blue";
    oContexte.fillRect(joueur.x, joueur.y, joueur.largeur, joueur.hauteur);

    // Afficher les contrôles
    oContexte.fillStyle = "black";
    oContexte.fillText("Flèches: Bouger | Espace: Sauter | Flèche Haut + Espace:
Tirer", 10, 20);
}

function tirerProjectile(x, y) {
    console.log("Tir depuis la position: " + x + ", " + y);
    // Ici on créerait un projectile
}
```